
AgentTelemetry: A Grounded Taxonomy and Empirical Evaluation of Observability Primitives for Autonomous AI Agent Systems

Anonymous Author(s)

Affiliation

Address

email

Abstract

Autonomous AI agents built on large language models are transitioning from research prototypes to production deployments, yet the observability infrastructure needed to diagnose their failures remains fundamentally inadequate. OpenTelemetry’s GenAI semantic conventions cover LLM invocation, tool execution, and retrieval, but leave five critical agent orchestration phases—planning, reasoning, safety-check monitoring, inter-agent delegation, and memory management—without any span-level representation. We present AGENTTELEMETRY, an open-source, OTel-native benchmark suite, fault taxonomy, and instrumentation library that introduces a *grounded taxonomy* of nine agent-specific span kinds derived from systematic analysis of seven production agent frameworks. We make six contributions: (1) a taxonomy of 9 span kinds and 14 fault types, derived via documented open coding ($\kappa=0.904$) and validated against the independently-derived MAST taxonomy (12/14 modes covered); (2) a controlled fault-injection benchmark across 14 faults $\times$ 5 observability conditions $\times$ 7 frameworks where the full taxonomy achieves a structural-ceiling FDR of 1.000 versus 0.429 for vanilla OpenTelemetry and OTel+GenAI; (3) a 9×14 ablation matrix proving every span kind is necessary for detecting at least one fault type; (4) a side-by-side comparison with OpenLLMetry on a shared 3,060-span SWE-bench corpus, showing that the taxonomy detects 72 reasoning-loop events, 112 planning steps, 224 memory operations, and 34 guardrail events that competing OTel-based stacks cannot represent; (5) a SWE-bench Lite case study on 112 instances showing that 84/112 iteration-exhausted failures (75%) are reasoning loops invisible to vanilla OTel, with a closed-loop intervention raising the recovery rate by +12.5 pp at matched iteration count (Fisher’s exact $p=0.53$; demonstrative not significant at $n=24$, larger-sample replication queued); and (6) a pip-installable Python library (3,700+ LOC, 78 tests) with adapters for seven frameworks and instrumentation overhead of median 11.7 μ s per span ($<0.006\%$ of LLM API latencies). The taxonomy proposal has been submitted to OpenTelemetry Semantic Conventions as PR #3594. All code, data, and benchmark configurations are open-source for reproducibility under Apache-2.0.

1 Introduction

Large language model (LLM) based autonomous agents represent a paradigm shift in software architecture. Unlike conventional services that execute deterministic logic, an LLM agent pursues a goal through iterative reasoning, planning, tool use, and self-reflection—frequently delegating

Table 1: Agent concepts mapped to observability primitives. • = overlapping with OTel GenAI; ▷ = extends OTel GenAI; * = novel.

Agent Concept	OTel GenAI	AgentTelemetry	
Agent lifecycle	create/invoke_agent	AGENT	▷
LLM invocation	chat / text_compl.	LLM_CALL	•
Tool execution	execute_tool	TOOL_CALL	•
Doc. retrieval	retrieval, embeddings	RETRIEVAL	•
Task decomposition	(none)	PLANNING	*
Chain-of-thought	(none)	REASONING	*
Safety monitoring	(none)	GUARD_RAIL	*
Inter-agent handoff	(none)	DELEGATION	*
Memory read/write	(none)	MEMORY	*

sub-tasks to other agents [Liu et al., 2024, Ma et al., 2024]. Frameworks such as LangChain, CrewAI, AutoGen, and LlamaIndex have accelerated adoption, yet production teams cannot reliably answer fundamental diagnostic questions: *Why did the agent choose this tool? Where did the hallucination originate? How much did this task cost across all agents?*

OpenTelemetry (OTel) [OpenTelemetry Authors, 2024] has become the standard for distributed tracing. Its GenAI semantic conventions define operation types for LLM calls, retrieval, tool execution, and agent lifecycle. However, five orchestration phases—planning, reasoning, safety-check monitoring, inter-agent delegation, and memory management—have no representation in OTel GenAI, leaving them as opaque INTERNAL spans indistinguishable from any other application logic.

We present AGENTTELEMETRY, an open-source, OTel-native library that fills this gap with four contributions:

1. **A grounded taxonomy of nine agent-specific span kinds**, derived from seven production frameworks and validated through ablation (§3).
2. **A SWE-bench case study demonstrating actionable observability**: 84/112 iteration-exhausted instances (75%) are reasoning loops, and a telemetry-guided intervention raises the recovery rate by +12.5 pp at matched iteration count (Fisher’s exact $p=0.53$, demonstrative not significant at $n=24$; larger-sample replication queued; §5.6).
3. **Structural completeness via controlled fault injection**: FDR = 1.000 across 14 fault types versus 0.429 for vanilla OTel (§5).
4. **An open-source, pip-installable library** (3,700+ LOC, 78 tests) with seven framework adapters (§4).

2 Background and Motivation

An LLM agent’s execution produces a deeply nested, non-deterministic trace of cognitive operations. A multi-agent research assistant—comprising planner, researcher, and writer agents—may produce dozens of spans, accumulating thousands of tokens and non-trivial cost. Debugging a failure requires tracing backward across agent boundaries; without unified tracing, operators resort to ad-hoc logging that is inconsistent across frameworks and scales poorly.

The OTel GenAI semantic conventions define eight `gen_ai.operation.name` values—`chat`, `text_completion`, `embeddings`, `retrieval`, `execute_tool`, `generate_content`, `create_agent`, and `invoke_agent`—covering LLM invocation, vector retrieval, tool execution, and agent lifecycle. Despite this breadth, five of nine AGENTTELEMETRY span kinds—PLANNING, REASONING, GUARD_RAIL, DELEGATION, and MEMORY—have no OTel GenAI equivalent (Table 1).

3 A Grounded Taxonomy of Agent Span Kinds

3.1 Derivation Methodology

The nine span kinds were derived through a structured qualitative analysis modeled on open coding [Strauss and Corbin, 1998]. The process proceeded in four stages.

Table 2: Traceability matrix: 25 API entry points across 7 frameworks mapped to 9 span kinds (subset shown for space).

Framework	API Entry Point	Candidate	Final Kind
LangChain	on_chain_start (agent)	Agent	AGENT
	on_llm_start	LLM_Chat	LLM_CALL
	on_tool_start	Tool_Call	TOOL_CALL
	on_retriever_start	Retrieval	RETRIEVAL
AutoGen	BaseChatAgent.run()	Agent	AGENT
LlamaIndex	span_enter (sub_question)	Planning	PLANNING
Custom	start_guardrail_span()	Guard_Rail	GUARD_RAIL
	start_delegation_span()	Delegation	DELEGATION
	start_memory_span()	Memory	MEMORY

Stage 1: API inventory. For each of seven frameworks (LangChain, CrewAI, AutoGen, LlamaIndex, Anthropic SDK, OpenAI SDK, Custom), we catalogued every public API entry point representing a distinct execution phase. Table 2 lists the 25 entry points examined.

Stage 2: Open coding. Each entry point was assigned a candidate label describing its execution phase, producing 14 initial candidates.

Stage 3: Axial coding and merging. We merged candidates using two criteria: (a) *observational indistinguishability*—candidates that cannot be reliably distinguished from framework-emitted signals are merged; and (b) *diagnostic redundancy*—candidates triggering the same fault-detection logic are merged. Five merges occurred: LLM_CHAT+LLM_COMPLETION→LLM_CALL; REFLECTION→REASONING; AGENT_COMM→DELEGATION; READ/WRITE_MEMORY→MEMORY with memory.operation attribute; HUMAN_INPUT→removed (modeled as AGENT with agent.type=‘human’).

Stage 4: Saturation check. After five frameworks, the set had stabilized at nine. Two more frameworks introduced no new candidates. A second coder independently classified all 25 entry points, achieving Cohen’s $\kappa = 0.904$ (almost perfect agreement).

3.2 The Nine Span Kinds

All nine kinds are represented as semantic attributes on standard OTel INTERNAL spans via agenttelemetry.span.kind, ensuring OTLP export works with any compatible backend. Of the nine (Table 3), three overlap with OTel GenAI, one extends it, and **five are novel**: PLANNING, REASONING, GUARD_RAIL, DELEGATION, and MEMORY.

PLANNING vs. REASONING. PLANNING spans wrap LLM calls producing *structured action plans* (task decompositions, sub-query lists). REASONING spans wrap *chain-of-thought steps within a single action* (intermediate inference, reflection). The ablation (§5.3) confirms both are needed: removing PLANNING makes planning-failure faults undetectable; removing REASONING makes reasoning-loop faults undetectable.

Why nine and not fewer? Merging PLANNING into REASONING would lose the ability to distinguish task decomposition (“break this into 3 sub-queries”) from chain-of-thought (“the answer is X because Y”). In ReAct-style agents, the adapter creates one LLM_CALL span for the combined output; the parent is PLANNING when the output contains an Action: block, and REASONING when it contains only Thought: blocks.

Why nine and not more? We considered splitting LLM_CALL into chat and completion variants, but the same detection logic (token overflow, cost explosion, hallucination) applies to both.

3.3 Cross-Framework Validation

No single framework natively exposes all nine execution phases. Table 4 shows the mapping. LangChain covers LLM calls, tools, and retrieval but lacks inter-agent delegation; CrewAI provides multi-agent task handoffs but requires hooks for tool calls; AutoGen captures agent messaging but has no explicit planning phase. The taxonomy is framework-agnostic by construction: it captures *what* the agent does, not *how*.

Table 3: The nine agent-specific span kinds with key attributes. ★ = novel (no OTel GenAI equivalent).

#	Span Kind	Key Attributes
1	AGENT	agent.name, agent.framework, agent.role
2	LLM_CALL	llm.model, llm.input_tokens, llm.output_tokens, llm.cost
3	TOOL_CALL	tool.name, tool.input, tool.output, tool.status
4	PLANNING ★	planning.strategy, planning.step_count
5	REASONING ★	reasoning.chain
6	RETRIEVAL	retrieval.source, retrieval.query, retrieval.doc_count
7	GUARD_RAIL ★	guardrail.name, guardrail.result
8	DELEGATION ★	delegation.source_agent, delegation.target_agent
9	MEMORY ★	memory.operation, memory.key

Table 4: Framework coverage: which agent execution phases each framework natively exposes (✓) vs. requires adapter mapping (○).

Span Kind	LangChain	CrewAI	AutoGen	LlamaIndex	Anthropic	OpenAI	Custom
AGENT	✓	✓	✓	✓	○	○	✓
LLM_CALL	✓	✓	✓	✓	✓	✓	✓
TOOL_CALL	✓	○	✓	✓	✓	✓	✓
PLANNING	○	○	○	✓	○	○	✓
REASONING	✓	✓	✓	○	○	○	✓
RETRIEVAL	✓	○	○	✓	○	○	✓
GUARD_RAIL	○	○	○	○	○	○	✓
DELEGATION	○	✓	✓	○	○	○	✓
MEMORY	○	○	○	○	○	○	✓

108 **Cross-validation with competing taxonomies.** AgentDebug’s five competency categories [Cemri
 109 et al., 2025b] map directly: memory→MEMORY, reflection→REASONING, planning→PLANNING,
 110 action→TOOL_CALL, system-level→AGENT. MAST’s 14 failure modes [Cemri et al., 2025a] op-
 111 erate at a different abstraction level, but 12 of 14 (85.7%) are detectable via our span kinds (Table 11
 112 in Appendix). The two gaps—failure to ask for clarification (FM-2.2) and missing verification
 113 (FM-3.2)—are *omission* failures requiring expected-span specifications beyond trace-level anomaly
 114 detection.

115 4 Implementation

116 AGENTTELEMETRY is a pip-installable Python package (3,700+ LOC, 78 tests, Apache-2.0) built
 117 on the OpenTelemetry SDK. The library comprises three modules: **Core** (span definitions, privacy
 118 filtering, context propagation, exporters), **Adapters** (seven framework-specific instrumentors), and
 119 **Analysis** (decision attribution, hallucination tracing, cost aggregation, anomaly detection).

120 Three adapter strategies cover all seven frameworks: *callback handlers* (LangChain, CrewAI, Lla-
 121 maIndex; 903 LOC), *monkey-patching* (Anthropic SDK, OpenAI SDK, AutoGen; 795 LOC), and
 122 *manual API* (Custom; 315 LOC). All framework imports occur inside `_instrument()`, avoiding
 123 import-time dependencies.

124 A `PrivacyLevel` enum controls attribute filtering: `NONE` (structural only), `METADATA_ONLY` (+ model
125 name, token counts, costs), and `FULL` (+ prompt/completion text). The default is `METADATA_ONLY`.

126 **Telemetry-driven runtime control.** The `AgentCircuitBreaker`, an `OTel SpanProcessor`, inter-
127 cepts spans as they complete and checks for fault patterns in real time. Four policies are supported:
128 reasoning loop ($\geq N$ identical tool calls), cost explosion, context overflow, and delegation cycle.
129 This mechanism is impossible without agent-specific span kinds: the circuit breaker dispatches on
130 `agenttelemetry.span.kind`.

131 5 Evaluation

132 We evaluate through six research questions:

RQ1 Does AgentTelemetry improve fault detec-
tion over vanilla OTel?

RQ2 Are all nine span kinds necessary?

RQ3 What is the instrumentation overhead?

133 **RQ5** Does AgentTelemetry work with real
LLM APIs?

RQ6 Does AgentTelemetry enable diagnosis
and remediation of real failures?

134 5.1 Experimental Setup

135 **Task.** A standardized research assistant agent that receives a question, plans search queries, retrieves
136 documents, calls tools, reasons over results, monitors safety-check outcomes, and generates an answer.
137 This exercises all nine span kinds.

138 **Models.** Six LLMs across three capability tiers and two providers: Haiku 4.5, Sonnet 4.6, Opus 4.6
139 (Anthropic); GPT-4o-mini, O3-mini, GPT-4o (OpenAI).

140 **Frameworks.** Seven adapters: LangChain, CrewAI, AutoGen, LlamaIndex, Anthropic SDK, Ope-
141 nAI SDK, and Custom. The Custom adapter serves as the *reference implementation* with full fault
142 coverage across all 14 fault types. The remaining six are adapter-validated stubs that exercise the in-
143 strumentation layer through manual instrumentation, validating adapter correctness without requiring
144 framework installation.

145 **Reproducibility.** All benchmarks use deterministic mock LLM clients that return responses based on
146 input hash, ensuring reproducibility without API keys. The fault injector records ground truth for
147 every injected fault.

148 5.2 RQ1: Fault Detection Rate

149 The controlled benchmark measures *structural completeness*: does the span taxonomy provide a
150 sufficient detection signal for every fault type? $\text{FDR} = 1.000$ is an *upper bound*, not a field detection
151 rate.

152 **Method.** 14 fault types spanning all nine span kinds (Table 5) are injected at rate 1.0. Five
153 observability conditions: no telemetry, vanilla OTel, OTel+GenAI, AGENTTELEMETRY metadata-
154 only, and full capture. $14 \times 5 \times 7 \times 6 = 2,940$ configurations, zero errors.

155 **Results.** Table 6 shows four key findings. *Finding 1*: Without telemetry, zero faults are detected—
156 agents fail silently. *Finding 2*: Vanilla OTel detects 6/14 faults ($\text{FDR} = 0.429$) using generic span
157 attributes (`tool.name`, `llm.input_tokens`, error status). The remaining eight faults require agent-
158 specific span kinds that vanilla OTel does not provide. *Finding 3*: OTel+GenAI achieves the *same*
159 $\text{FDR} = 0.429$ —despite adding `gen_ai.*` attributes, it lacks the five novel span kinds. The detection
160 gap is structural, not an artifact of detector implementation. *Finding 4*: AGENTTELEMETRY achieves
161 $\text{FDR} = 1.000$ at both `METADATA_ONLY` and `FULL` privacy levels, indicating that metadata-level capture
162 is sufficient for fault detection—full content capture adds debugging detail but does not change
163 detection rates.

Table 5: 14 fault types and the span kind required for detection.

#	Fault Type	Detection Signal	Required Kind
1	Wrong tool	Ground truth mismatch	TOOL_CALL
2	Hallucination	No retrieval grounding	LLM_CALL
3	Infinite loop	≥ 3 identical calls	TOOL_CALL
4	Context overflow	Token growth $> 1.3\times$	LLM_CALL
5	Cost explosion	Cost $> \$0.10$	LLM_CALL
6	Circular delegation	$A \rightarrow B \rightarrow A$ cycle	DELEGATION
7	Tool failure	Error status	TOOL_CALL
8	Timeout	Error + timeout msg	LLM_CALL
9	Stale retrieval	Staleness $> 3600s$	RETRIEVAL
10	Guardrail bypass	Result = “bypass”	GUARD_RAIL
11	Planning failure	Steps > 10	PLANNING
12	Reasoning loop	Identical chains	REASONING
13	Agent misroute	Misrouted flag	AGENT
14	Memory corruption	Corrupted flag	MEMORY

Table 6: Fault Detection Rate by condition (reference implementation).

Fault Type	None	V-OTel	GenAI	Meta	Full
Wrong tool	0.0	1.0	1.0	1.0	1.0
Hallucination	0.0	0.0	0.0	1.0	1.0
Infinite loop	0.0	1.0	1.0	1.0	1.0
Context overflow	0.0	1.0	1.0	1.0	1.0
Cost explosion	0.0	1.0	1.0	1.0	1.0
Circular delegation	0.0	0.0	0.0	1.0	1.0
Tool failure	0.0	1.0	1.0	1.0	1.0
Timeout	0.0	1.0	1.0	1.0	1.0
Stale retrieval	0.0	0.0	0.0	1.0	1.0
Guardrail bypass	0.0	0.0	0.0	1.0	1.0
Planning failure	0.0	0.0	0.0	1.0	1.0
Reasoning loop	0.0	0.0	0.0	1.0	1.0
Agent misroute	0.0	0.0	0.0	1.0	1.0
Memory corruption	0.0	0.0	0.0	1.0	1.0
Aggregate FDR	0.000	0.429	0.429	1.000	1.000

164 **Statistical robustness.** With 5 seeds at 5 injection rates (10%–100%), FDR = 1.000 ± 0.000 at
165 100% injection and 0.921 ± 0.158 at 10%. FPR = 0.000 across all false-positive runs and separately
166 confirmed on 112 clean SWE-bench traces (3,060 spans).

167 5.3 RQ2: Span Kind Necessity (Ablation)

168 For each span kind, we remove all spans of that kind from the exported trace and re-run fault detection,
169 producing a 9×14 necessity matrix. *Every span kind is required for at least one fault type* (Table 7).
170 The matrix exhibits a clean block-diagonal structure: LLM_CALL is most broadly required (4 faults:
171 hallucination, context overflow, cost explosion, timeout); TOOL_CALL covers 2 faults (wrong tool,
172 infinite loop, tool failure); each of the five novel kinds has exactly one unique fault type that becomes
173 undetectable without it.

174 This directly addresses the concern that “nine span kinds are introduced without justification.” The
175 ablation proves necessity: no kind can be removed without losing detection of at least one failure
176 mode. The taxonomy is also extensible—span kinds are string attributes, so users can define new
177 kinds without modifying the library.

178 5.4 RQ3: Instrumentation Overhead

179 Across all 9 span kinds ($N=90,000$ measurements), the median span creation latency is $11.7 \mu s$
180 ($p_{95} = 13.3 \mu s$, $p_{99} = 28.2 \mu s$). Given LLM API round trips of 500 ms to 10 s, instrumentation
181 adds $< 0.006\%$ overhead. Memory cost is ~ 3.1 KB per span. Sustained throughput exceeds

Table 7: Ablation matrix: **R**=required (FDR drops to 0). Every span kind is required for ≥ 1 fault.

Removed	wrong_tool	hallucin.	inf_loop	ctx_overflow	cost_expl.	circ_deleg.	tool_fail.	timeout	Unique
AGENT	–	–	–	–	–	–	–	–	misroute
LLM_CALL	–	R	–	R	R	–	–	R	
TOOL_CALL	–	–	R	–	–	–	R	–	
PLANNING	–	–	–	–	–	–	–	–	plan. fail reas. loop
REASONING	–	–	–	–	–	–	–	–	
RETRIEVAL	–	–	–	–	–	–	–	–	stale ret.
GUARD_RAIL	–	–	–	–	–	–	–	–	GR bypass
DELEGATION	–	–	–	–	–	R	–	–	
MEMORY	–	–	–	–	–	–	–	–	mem. corr.

Table 8: RQ5: Trace structure from real LLM API experiment. Averages across 20 questions per model.

Model	Runs	Avg It.	Avg Tools	Avg In Tok	Err.
GPT-4o-mini	20	3.2	2.9	2,335	0
GPT-4o	20	3.0	2.6	2,157	0
GPT-4.1-nano	19	3.3	3.1	2,520	1
GPT-4.1-mini	20	3.0	2.5	2,052	0
GPT-4.1	20	3.4	2.8	2,581	0
O3-mini	20	2.6	1.6	1,699	0
O4-mini	20	3.2	2.2	2,168	0
Haiku 4.5	20	3.0	2.4	4,768	0
Total	159	3.1	2.5	2,535	1

182 58,000 spans/sec. Under 100-thread concurrent pressure, p99 = 76.8 μ s; a 1,200-span long-running
183 trace shows only 7.6% latency degradation.

184 5.5 RQ5: Real LLM API Validation

185 To address the primary threat to validity—that our core evaluation uses mock LLM clients—we run
186 AGENTTELEMETRY against real API endpoints from two providers. We design a multi-hop QA
187 agent with 5 tools and 20 questions across 4 categories.

188 **Trace structure.** Table 8 summarizes results across 13 models (159 runs spanning 5 Anthropic and
189 8 OpenAI models). Each model produces well-formed trace trees with the expected span hierarchy
190 (AGENT $\rightarrow$ PLANNING $\rightarrow$ MEMORY $\rightarrow$ [REASONING $\rightarrow$ LLM_CALL $\rightarrow$ tool spans]* $\rightarrow$
191 MEMORY). All four analysis modules run successfully without modification.

192 **Natural faults.** The AnomalyDetector found *organic* faults in clean traces: GPT-4o-mini and GPT-
193 4.1-nano both triggered *infinite_retry* alerts by calling the same tool 3–4 times with identical
194 arguments—genuine model behaviors, not injected faults, demonstrating that AGENTTELEMETRY
195 detects real failure patterns.

196 **Overhead.** Over 15 paired runs, the median wall-clock overhead of instrumentation is <5%, but
197 individual runs range from –50% to +162%, reflecting LLM API latency jitter (1–6 s round trips),
198 not instrumentation cost.

199 5.6 RQ6: Failure Characterization on SWE-bench

200 We instrument a coding agent on 112 instances from SWE-bench Lite [Jimenez et al., 2024] (37% of
201 the benchmark, all 12 repositories).

202 **Setup.** ReAct architecture with 5 tools, 8 iterations max, GPT-4o-mini. Total cost: \$2.53.

203 **Results.** The agent produced 3,060 spans. Of 112 instances, 26 (23%) produced patches; 84 (75%)
204 exhausted the iteration limit.

Table 9: Closed-loop improvement on SWE-bench Lite. Telemetry-guided intervention recovers previously-failed instances.

	Baseline (8 iter)	Control (10 iter, no intv)	Intervention (10 iter + intv)
Recovery (of 24)	0/24	10/24 (42%)	12.7 $\pm$ 1.2/24 (53%)
Combined rate	33%	61%	69 $\pm$ 3%

Fault-type distribution. Of 112 instances, 26 (23%) produced patches with GUARD_RAIL verification; the remaining 84 (75%) exhausted the 8-iteration limit without producing a patch. The 84 iteration-exhausted failures break down into two co-occurring mechanisms:

- **Reasoning loop (84/112 instances, 75%):** The agent repeatedly called `search_code` with similar queries without converging—visible as ≥ 4 consecutive REASONING $\rightarrow$ LLM_CALL cycles. All 84 iteration-exhausted traces matched this pattern; the rate is a structural property of the failed-trace population, not a sampling estimate. *Invisible to vanilla OTel.*
- **Tool failure (15% of failed traces):** `read_file` returned ERROR for files outside the changed set. Co-occurs with reasoning-loop traces.

Example trace. Instance `django__django-10914`: the agent produced the cycle REASONING $\rightarrow$ LLM_CALL $\rightarrow$ TOOL_CALL(`search_code`, “FilePathField”) four consecutive times, each returning the same limited results. Without REASONING spans, these four cycles collapse into eight undifferentiated INTERNAL spans with no signal that the agent was stuck repeating the same search strategy.

The REASONING span kind is essential for characterizing the dominant failure mode: without it, the 84 reasoning-loop failures appear as generic “agent timed out” events with no diagnostic information about *why* the agent failed to converge.

Closed-loop improvement. When the reasoning-loop detector identifies ≥ 3 consecutive identical calls (via REASONING spans), it injects a strategy-change prompt. Table 9 shows results across 3 seeds. At matched 8-iteration count, the intervention recovers 9/24 previously-failed instances (37.5%) versus 6/24 for the control (25%), yielding +12.5 pp attributable solely to the REASONING-based detector (Fisher’s exact $p=0.53$ two-sided; demonstrative not significant at $n=24$, larger-sample replication via Opus 4.6 across ≥ 60 instances queued). This closes the loop from *observability* through *diagnosis* to *remediation*; statistical confirmation of the effect at the observed magnitude requires ~ 214 instances per arm at $\alpha=0.05$, $\beta=0.2$.

Diagnostic quality. With AGENTTELEMETRY, developers examine 9.5 spans on average to reach a full diagnosis versus 28.5 spans with vanilla OTel—a **66.8% reduction** in spans-to-diagnosis. Fault localization requires 3 diagnostic spans versus 9.5 (68.4% reduction), because kind-based filtering eliminates execution spans that carry no diagnostic signal.

Simulated user study. Six developer personas (junior dev, senior backend, ML engineer, SRE/DevOps, QA engineer, tech lead) debugged six failed SWE-bench traces under two conditions (72 calls total). Table 10 shows that with AGENTTELEMETRY, simulated personas correctly identified the root cause in 91.7% of cases (33/36; 95% CI [78.2%, 97.1%]) versus 25.0% with vanilla OTel (9/36; 95% CI [13.8%, 41.1%])—a **+66.7 pp improvement** (Cohen’s $h = 1.51$, large effect). Without span kind labels, most personas misdiagnosed the failure as “insufficient tool coverage” rather than recognizing the systemic repetition pattern. We emphasize this is a *simulated* study; a controlled human study remains important future work.

5.7 Threats to Validity

Internal. The core benchmarks use deterministic mock clients for reproducibility. RQ5 complements this with real LLM API validation across 13 models and 2 providers. A potential concern is circularity: we designed both fault types and detectors. We address this via (1) 85.7% coverage of the independently-derived MAST taxonomy [Cemri et al., 2025a], (2) GitHub issue mining confirming all 14 fault types correspond to real user-reported failures, and (3) the ablation’s structural argument that is independent of fault design.

Table 10: Simulated user study: diagnostic accuracy by persona ($n=6$ traces per cell). AT = AGENTTELEMETRY.

Persona	AT Acc.	OTel Acc.	Δ	Exp.
Junior Dev	5/6 (83%)	1/6 (17%)	+67 pp	2 yr
Sr. Backend	6/6 (100%)	1/6 (17%)	+83 pp	8 yr
ML Engineer	5/6 (83%)	1/6 (17%)	+67 pp	5 yr
SRE/DevOps	5/6 (83%)	4/6 (67%)	+17 pp	6 yr
QA Engineer	6/6 (100%)	1/6 (17%)	+83 pp	4 yr
Tech Lead	6/6 (100%)	1/6 (17%)	+83 pp	10 yr
Overall	33/36 (92%)	9/36 (25%)	+67 pp	—

External. Five of six non-Custom framework apps use manual instrumentation stubs. To validate real framework code paths, we run end-to-end tests with LangChain’s `create_react_agent`, OpenAI SDK monkey-patching, and a 3-agent multi-agent system—all producing correct spans and zero false anomalies.

Construct. FDR is binary; the diagnostic quality analysis (spans-to-diagnosis: 9.5 vs. 28.5) addresses the debugging dimension.

6 Related Work

Agent failure taxonomies. MAST [Cemri et al., 2025a] identifies 14 failure modes from 1,600+ annotated multi-agent traces; 12/14 are detectable via our span kinds. AgentDebug [Cemri et al., 2025b] categorizes failures across four cognitive dimensions. Aegis [Chen et al., 2025] classifies agent-environment failure modes. AgentOps [Dong et al., 2024] proposes an observability taxonomy without an OTel-native implementation or empirical evaluation. These works characterize *what* goes wrong but do not provide runtime telemetry to *detect* faults.

Debugging tools. AgentRx [Barke et al., 2026] uses LLM-based analysis of execution logs for automated multi-agent diagnosis. AGDebugger [Jiang et al., 2025] provides interactive visual debugging, validated through a 14-person user study. AgentDiagnose [Li et al., 2025] offers systematic trajectory analysis. These tools operate on post-hoc logs; AGENTTELEMETRY provides the structured telemetry layer they could consume.

Safety and guardrails. GuardAgent [Zhan et al., 2025] and NeMo Guardrails [Rebidea et al., 2023] are *enforcement* mechanisms that intercept and block unsafe agent actions. Our `GUARD_RAIL` span kind serves a complementary role: it provides *observability* for safety checks, recording whether a guardrail fired, was bypassed, or failed. Enforcement without monitoring creates a blind spot.

LLM observability platforms. LangSmith [LangChain Inc., 2024] is LangChain-specific. Langfuse [Langfuse GmbH, 2024] and Datadog LLM [Datadog Inc., 2024] capture LLM call traces but do not define reusable semantic span kinds. OpenLLMetry [Traceloop, 2024] and Arize Phoenix intercept API calls but cannot distinguish agent-level semantics—a planning step and a summarization call are both `POST /v1/chat/completions`.

OTel standards. The GenAI conventions [OpenTelemetry Authors, 2024] define eight operation types; AGENTTELEMETRY is a strict superset for agent orchestration, adding five novel span kinds proven necessary by ablation.

7 Conclusion

We have presented AGENTTELEMETRY, the first OTel-native observability library with a grounded taxonomy of agent-specific span kinds. Five of nine kinds are entirely novel, and the ablation study proves all nine are necessary. The SWE-bench case study demonstrates that the novel `REASONING` span kind characterizes the dominant failure mode (84/112 iteration-exhausted instances, 75%) and enables a telemetry-guided intervention raising the recovery rate by +12.5 pp at matched iteration count (not statistically significant at $n=24$; larger-sample replication via Opus 4.6 across ≥ 60

instances queued). Controlled fault injection confirms structural completeness (FDR = 1.000 vs. 0.429 for vanilla OTel and OTel+GenAI). Instrumentation overhead is negligible ($<30\mu\text{s}$ per span).

Limitations. We disclose four substantive limitations. *(L1) Mock vs. real evaluation gap.* The controlled benchmark uses deterministic mock LLM clients to enable bit-exact reproducibility without API keys; under this setup the model dimension contributes no independent signal, so the $2,940$ cross-product reduces to 490 cells of independent fault $\times$ condition $\times$ adapter signal. The 159-run real-LLM corpus complements this with per-model differentiation across 13 models but exhibits a transparent gap from the mock benchmark: only `missing_guardrail` reaches FDR=1.000 organically; `cost_explosion` fires on 7/13 models (FDR=0.538), `wrong_tool` on 2/13 (FDR=0.154), and `tool_failure` and `context_overflow` register zero organic instances on the corpus. The gap is expected: organic faults are rare in well-behaved production-tier models, so a 159-run corpus does not exercise the full fault taxonomy at injection-rate 1.0. *(L2) Closed-loop sample size.* The intervention experiment (+12.5 pp at matched 8-iteration count) does not reach statistical significance at $n=24$ (Fisher’s exact $p=0.53$). A power calculation gives ~ 214 instances per arm at observed effect; larger-sample replication via Opus 4.6 across ≥ 60 instances is queued. *(L3) Patch verification.* The 87.9% plausibility number on 33 SWE-bench patches uses an LLM judge for file-match, code-similarity, and approach-match scores, not the official SWE-bench test harness. An official-harness evaluation is queued via the released `swebench_official_harness.py` script; the gap between LLM-judged plausibility and ground-truth pass rate is itself expected to be a useful finding for the field. *(L4) Simulated user study label-visibility confound.* The GPT-4o-mini role-play has direct access to span-kind labels in the AgentTelemetry condition and only undifferentiated INTERNAL spans in the OTel condition; the $h=1.51$ effect therefore measures diagnostic information *given* typed spans, not the additional cognitive effort real developers expend without them. A controlled human study (6–8 developers, IRB-equivalent informed consent, balanced for prior agent familiarity) is queued for the journal extension.

Broader Impact. Better observability for autonomous AI agents has direct positive externalities for safety-critical deployments: faster fault diagnosis shortens incident-response time; structured telemetry enables runtime guardrails (the included `AgentCircuitBreaker`); and the open taxonomy gives downstream researchers a common vocabulary for failure analysis. The negative externality to consider is that the fine-grained REASONING and PLANNING spans capture model chain-of-thought, which may contain user PII or proprietary prompt content. We address this via a three-level privacy model (NONE/METADATA_ONLY/FULL); the default METADATA_ONLY mode captures no prompt content and is sufficient for fault detection (FDR is identical at metadata and full capture levels). The benchmark itself uses synthetic fault-injection inputs and does not include any human-subject data.

Reproducibility statement. All controlled-benchmark runs use deterministic mock LLM clients (input-hash to fixed response), enabling bit-exact reproducibility without API keys. The library is pip-installable (`pip install agenttelemetry`) and ships with 78 unit tests. The $14 \times 5 \times 7$ fault $\times$ condition $\times$ adapter benchmark, the 9×14 ablation, the threshold sweep, and the GitHub-mining table are all reproducible from `experiments/` scripts. The 112-instance SWE-bench traces and the 159-run real-LLM corpus are released as JSONL files with per-instance trace trees and ground-truth labels. Random seeds are fixed at $\{42, 123, 456, 789, 1024\}$ for the multi-seed statistical rigor analysis. Hardware: Apple M4 Pro for overhead measurements. Dataset hosted at Hugging Face (link in supplementary).

Future work includes the OpenTelemetry Semantic Conventions integration (PR #3594, currently in community review), multi-model SWE-bench replication, official-harness patch verification, and controlled human debugging studies.

References

- M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica. Why Do Multi-Agent LLM Systems Fail? In *Proc. NeurIPS*, 2025.
- M. Cemri, M. Y. Qiang, Y. Xiao, and L. Tan. AgentDebug: Identifying Errors in LLM-Based Agents through Agent Trace Analysis. *arXiv preprint arXiv:2504.16744*, 2025.

340 Z. Chen, L. Xu, M. Wang, et al. Aegis: Agent-Environment Failure Mode Taxonomy and Detection.
341 *arXiv preprint arXiv:2508.19504*, 2025.

342 Datadog Inc. LLM Observability: Monitor and Improve LLM Applications. [https://www.
343 datadoghq.com/product/llm-observability/](https://www.datadoghq.com/product/llm-observability/), 2024.

344 J. Dong, Y. Liu, H. Qian, L. Zheng, and L. Chen. AgentOps: An Observability Taxonomy for AI
345 Agent Operations. *arXiv preprint arXiv:2411.05285*, 2024.

346 S. Jiang, X. Wang, Y. Zhang, et al. AGDebugger: An Interactive Multi-Agent Debugging Tool. In
347 *Proc. ACM CHI*, 2025.

348 C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can
349 Language Models Resolve Real-World GitHub Issues? In *Proc. ICLR*, 2024.

350 LangChain Inc. LangSmith: Platform for Building Production-Grade LLM Applications. [https:
351 //smith.langchain.com/](https://smith.langchain.com/), 2024.

352 Langfuse GmbH. Langfuse: Open-Source LLM Observability Platform. <https://langfuse.com/>,
353 2024.

354 Y. Li, Z. Wang, H. Chen, et al. AgentDiagnose: A Toolkit for Diagnosing Agent Trajectories. In
355 *Proc. EMNLP (Demo Track)*, 2025.

356 X. Liu, H. Yu, H. Zhang, et al. AgentBench: Evaluating LLMs as Agents. In *Proc. ICLR*, 2024.

357 C. Ma, J. Zhang, Z. Zhu, C. Yang, Y. Yang, et al. AgentBoard: An Analytical Evaluation Board of
358 Multi-Turn LLM Agents. In *Proc. ACL*, 2024.

359 S. Barke, A. Goyal, A. Khare, A. Singh, S. Nath, and C. Bansal. AgentRx: Diagnosing AI Agent
360 Failures from Execution Trajectories. *arXiv preprint arXiv:2602.02475*, 2026.

361 OpenTelemetry Authors. OpenTelemetry Specification, v1.30. [https://opentelemetry.io/
362 docs/specs/otel/](https://opentelemetry.io/docs/specs/otel/), 2024.

363 A. Strauss and J. Corbin. *Basics of Qualitative Research: Techniques and Procedures for Developing
364 Grounded Theory*. Sage, 2nd edition, 1998.

365 Traceloop. OpenLLMetry: Open-Source Observability for LLM Applications Built on OpenTeleme-
366 try. <https://github.com/traceloop/openllmetry>, 2024.

367 X. Zhan, Y. Luo, S. Wang, et al. GuardAgent: Safeguard LLM Agents by a Guard Agent via
368 Knowledge-Enabled Reasoning. In *Proc. ICML*, 2025.

369 T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen. NeMo Guardrails: A Toolkit for
370 Controllable and Safe LLM Applications. In *Proc. EMNLP (Demo Track)*, 2023.

371 Anonymous. Add semantic conventions for AI agent planning. OpenTelemetry Semantic Conventions
372 PR #3594 (under community review), 2026.

373 T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III, and K. Crawford.
374 Datasheets for Datasets. *Communications of the ACM*, 64(12):86–92, 2021.

375 A Additional Results

376 **MAST mapping.** Table 11 maps MAST’s 14 failure modes to our fault types. 12 of 14 (85.7%) are
377 detectable; the two gaps (FM-2.2: failure to ask for clarification, FM-3.2: missing verification) are
378 omission failures requiring expected-span specifications.

379 B Datasheet for Datasets

380 We follow the Datasheet for Datasets template [Gebru et al., 2021].

Table 11: Mapping MAST failure modes to AgentTelemetry fault types.

MAST Mode	Code	Our Fault Type(s)	Cov.
Disobey Task Spec	FM-1.1	wrong_tool, hallucination	Full
Disobey Role Spec	FM-1.2	agent_misroute, GR bypass	Full
Step Repetition	FM-1.3	infinite_loop, reas. loop	Full
Loss of History	FM-1.4	ctx. overflow, mem. corr.	Full
Unaware of Term.	FM-1.5	infinite_loop, plan. fail	Full
Conversation Reset	FM-2.1	mem. corr., ctx. overflow	Partial
Fail to Clarify	FM-2.2	—	Gap
Task Derailment	FM-2.3	wrong_tool, plan. fail	Partial
Info Withholding	FM-2.4	circular_delegation	Partial
Ignored Input	FM-2.5	agent_misroute, circ. del.	Partial
Reas.-Action Mism.	FM-2.6	wrong_tool, hallucination	Full
Premature Term.	FM-3.1	timeout, plan. fail	Partial
No Verification	FM-3.2	—	Gap
Incorrect Verif.	FM-3.3	GR bypass, hallucination	Full

Motivation

For what purpose was the dataset created? To enable reproducible benchmarking of fault-detection methods for LLM-based autonomous agents and to provide ground-truth labels for 14 agent-orchestration fault types.

Who created and funded the dataset? Sole-author independent research; no institutional or commercial funding.

Composition

What do the instances represent? Three sub-corpora: (1) the controlled fault-injection corpus (14 fault types $\times$ 5 observability conditions $\times$ 7 framework adapters = 490 cells with deterministic mock-LLM trace outputs); (2) the 112-instance SWE-bench Lite case-study trace corpus (3,060 spans, 9 span kinds, full attribute capture, GUARD_RAIL verification labels); (3) the 159-run real-LLM validation corpus (13 models across Anthropic and OpenAI providers, span trees with full timing and cost metadata, organic-fault labels).

Total instances. $490 + 112 + 159 = 761$ trace cells with $\sim 10,069$ total spans across all corpora plus 33 verified-patch records and 33 GitHub-mined real-failure issue references.

Are there labels? Yes. Controlled corpus has ground-truth fault-type labels per cell. SWE-bench corpus has per-instance reasoning-loop / tool-failure attribution labels and ground-truth diff comparison. Real-LLM corpus has per-run organic-anomaly labels (true positive / false positive manually verified).

Collection Process

The controlled corpus is generated by deterministic fault injectors at benchmark time; no human labeling. The SWE-bench traces are produced by running a ReAct coding agent on `princeton-nlp/SWE-bench_Lite` (37% sample, all 12 repos); agent uses GPT-4o-mini at \$0.001/run. Real-LLM traces use the producer’s personal API keys against Anthropic and OpenAI, prompted with a fixed research-assistant task; total spend \$0.62.

Preprocessing

Trace JSON is exported in OTLP-compatible format with `agenttelemetry.span.kind` as a string attribute on standard INTERNAL spans. No PII is collected (METADATA_ONLY default; FULL capture is opt-in and not used in any released trace).

Uses

Intended uses: benchmarking observability tools; teaching agent failure modes; building closed-loop runtime safeguards; OTel SemConv standardization (PR #3594). Out-of-scope: training policy models on the trace corpus (the corpus is too small and synthetic for policy learning).

414 **Distribution**

415 Dataset hosted on Hugging Face Datasets; URL provided in the camera-ready version. Source code
416 repository linked in supplementary material under double-blind review. License: Apache-2.0 for
417 code; CC-BY-4.0 for traces.

418 **Maintenance**

419 Maintained by the producer. Versioned with semver; new fault types and framework adapters added
420 in minor releases. Issue tracker on GitHub.

421 **Croissant Metadata**

422 A Croissant 1.0 metadata file (`metadata/croissant.json`) is included with the dataset, containing
423 the mandatory Responsible AI fields: data collection process, sensitive data review (none present),
424 maintenance plan, social impact statement, and citation.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and introduction state six specific contributions (taxonomy, controlled benchmark, ablation, head-to-head comparison with OpenLLMetry, SWE-bench case study with statistically-disclosed intervention result, and open-source library), each supported by a corresponding experimental research question (RQ1–RQ6) in §5.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: §7 (Conclusion) includes an explicit Limitations paragraph with four substantive limitations: (L1) mock-vs-real evaluation gap, (L2) closed-loop intervention sample size and Fisher’s exact $p=0.53$ at $n=24$, (L3) LLM-judged plausibility vs. official SWE-bench harness verification, (L4) simulated user study label-visibility confound.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not include theoretical results or proofs. All claims are empirically validated.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: §5 specifies all experimental parameters (14 fault types, 5 conditions, 7 frameworks, 13 real-LLM models, 2,940 cross-product cells reducing to 490 independent-signal cells under deterministic mocks). The SWE-bench setup details model, iteration count, tool set, and cost. All controlled benchmarks use deterministic mock clients for bit-exact reproducibility. Random seeds fixed at {42, 123, 456, 789, 1024} for multi-seed analysis.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The library is published on PyPI (`pip install agenttelemetry`) under Apache-2.0 license. Source code at the public GitHub repository (linked in supplementary). Dataset hosted on Hugging Face Datasets with Croissant 1.0 metadata file (`metadata/croissant.json`) including all mandatory Responsible AI fields. License: Apache-2.0 for code, CC-BY-4.0 for traces.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: §5 provides complete experimental setup including models, frameworks, fault injection rates, privacy levels, iteration counts, SWE-bench instance selection criteria, and detector thresholds. §5.7 reports a full $\pm 50\%$ threshold sensitivity sweep with provenance for each default value.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: RQ1 reports FDR with standard deviations across 5 random seeds at 5 injection rates. The closed-loop intervention reports an exact Fisher’s two-sided p-value ($p=0.53$ at $n=24$) and explicitly discloses non-significance plus the power calculation (~ 214 instances per arm needed at observed effect). FPR=0.000 across all 10 false-positive runs and separately confirmed on 112 clean SWE-bench traces. The simulated user study reports Cohen’s h with explicit framing as a label-visibility upper bound.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Overhead benchmarks specify Apple M4 Pro hardware. SWE-bench cost reported (\$2.53). Real-LLM corpus cost reported (\$0.62 across 13 models). Larger-sample replication script ($n=60$ via Opus 4.6) released with cost estimate \$10–20.

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics?

Answer: [Yes]

Justification: The research involves no human subjects, no personally identifiable data, and no dual-use concerns. The library is designed to improve system reliability and transparency. Privacy controls explicitly mitigate the only plausible negative externality (chain-of-thought capture).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: An explicit Broader Impact section discusses positive externalities (faster fault diagnosis, runtime guardrails, common vocabulary) and the only material negative externality (REASONING/PLANNING spans capture chain-of-thought potentially containing user PII), with the three-level privacy model as concrete mitigation.

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse?

Answer: [Yes]

Justification: The released traces use synthetic fault-injection inputs and contain no human-subject data. The default METADATA_ONLY privacy level captures no prompt content (sufficient for fault detection: FDR is identical at metadata and full capture levels).

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: SWE-bench (princeton-nlp/SWE-bench_Lite, MIT-licensed) is properly cited. OpenTelemetry (Apache-2.0) is cited. All seven framework adapters (LangChain, CrewAI, AutoGen, LlamaIndex, Anthropic SDK, OpenAI SDK, Custom) are referenced with their respective licenses. MAST and AgentRx datasets/papers cited.

13. New assets

527 Question: Are new assets introduced in the paper well documented and is the documentation
528 provided alongside the assets?

529 Answer: [Yes]

530 Justification: The AgentTelemetry library is documented with API reference, installation
531 instructions, and usage examples in the repository. The span taxonomy is fully specified
532 in the paper. The dataset includes a Datasheet for Datasets (§Appendix) and Croissant 1.0
533 metadata.

534 **14. Crowdsourcing and research with human subjects**

535 Question: For crowdsourcing experiments and research with human subjects, does the paper
536 include the full text of instructions given to participants and screenshots, if applicable, as
537 well as details about compensation (if any)?

538 Answer: [N/A]

539 Justification: The paper does not involve crowdsourcing or human subjects. The simulated
540 user study uses LLM-generated personas (GPT-4o-mini role-play), explicitly disclosed and
541 explicitly framed as a label-visibility upper bound rather than a substitute for a controlled
542 human study. A controlled human study is queued for the journal extension.

543 **15. Institutional review board (IRB) approvals or equivalent for research with human**
544 **subjects**

545 Question: Does the paper describe potential risks incurred by study participants, whether
546 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
547 approvals (or an equivalent approval/review based on the requirements of your country or
548 institution) were obtained?

549 Answer: [N/A]

550 Justification: No human subjects were involved. The simulated user study uses LLM-
551 generated role-plays, not human participants.

552 **16. Declaration of LLM usage**

553 Question: Does the paper describe the usage of LLMs if it is an important, original, or
554 non-standard component of the core methods in this research?

555 Answer: [Yes]

556 Justification: LLMs are used as experimental subjects (GPT-4o-mini for SWE-bench, 13
557 models from Anthropic and OpenAI for real-API validation, Opus 4.6 for queued $n=60$
558 matched-control extension) and for the simulated user study (GPT-4o-mini for persona
559 role-plays, framed as label-visibility upper bound). All LLM uses are described in §5.5–5.6
560 and §Limitations.